



汇编语言

4 汇编语言程序员视角的x86处理器

张青

ieqzhang@zzu.edu.cn

教学内容



- 复习x86-64处理器

教材章节：3.1-3.4

慕课：4.1-4.4

- ▶ 寄存器

- ▶ 工作方式与存储模型

- ▶ 逻辑地址与物理地址转换

- 操作数寻址方式

教材章节：3.5

慕课：4.5-4.6

- ▶ 寻址方式

- 传送和运算类指令导学

教材章节：4.1-4.3

慕课：5.1-5.4

- ▶ 指令格式

- ▶ 指令助记符、功能

- ▶ 对标志位的影响

学习目标



- 能说出x86-64处理器中寄存器的名字和作用
- 能阐述逻辑地址、物理地址的含义及其转换方法，并可以用生活中的例子对各种地址进行解释说明
- 能说出x86-64处理器存储模型名字，并阐述其特点
- 能阐述x86-64处理器工作方式与存储模型之间的关系
- 能够阐述计算机中数据的基本单位以及他们之间的关系
- 能够阐述各种寻址方式的含义及其在汇编语言中的表示方式
- 理解汇编语言程序员视角的处理器结构，培养系统观



邀请码: 12570104

APP首页右上角输入



该邀请码2026年08月26日前有效

计算机类2024级3班4班

复习x86-64处理器

邀请码: 97974770

APP首页右上角输入



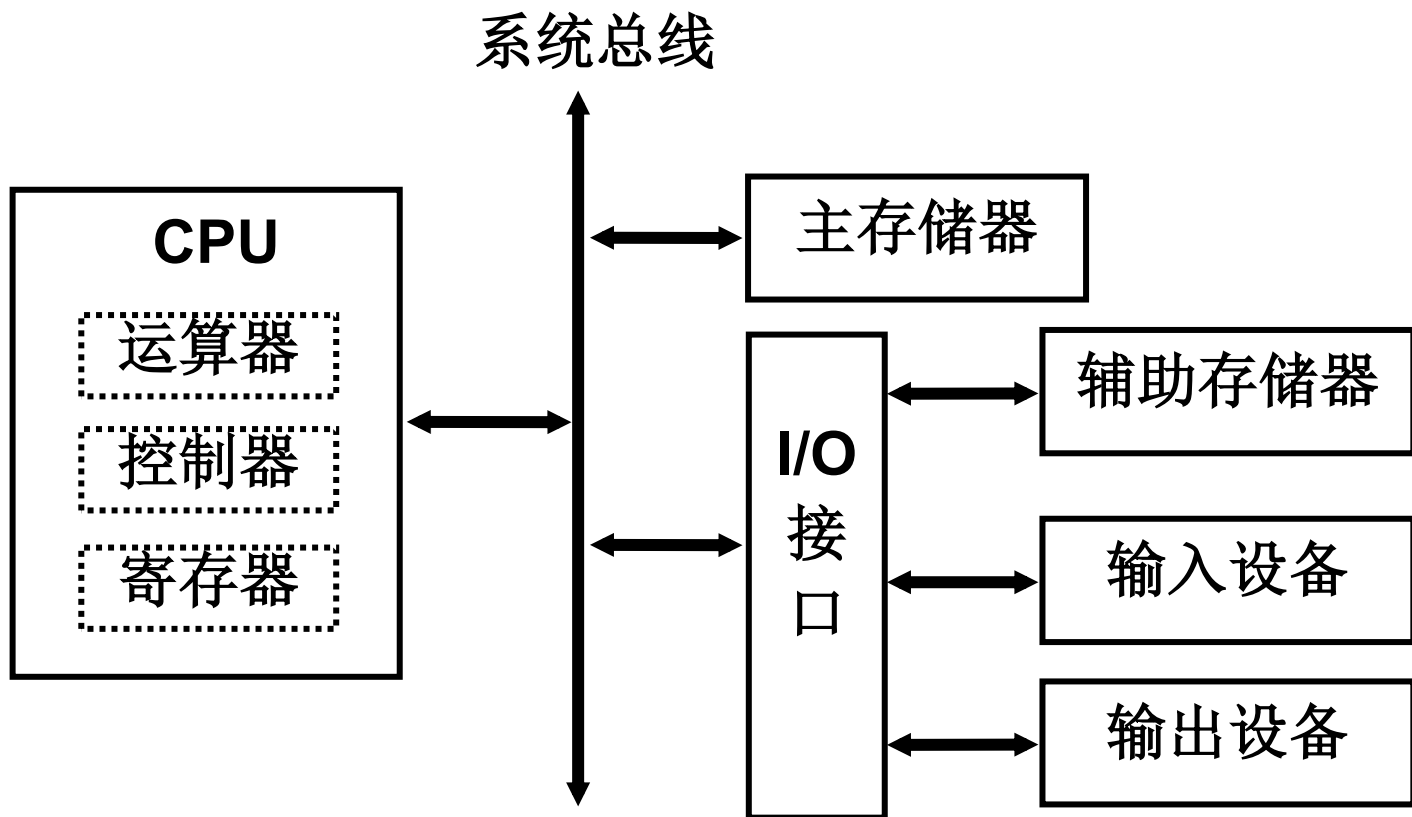
复习x86-64处理器



该邀请码2026年08月26日前有效

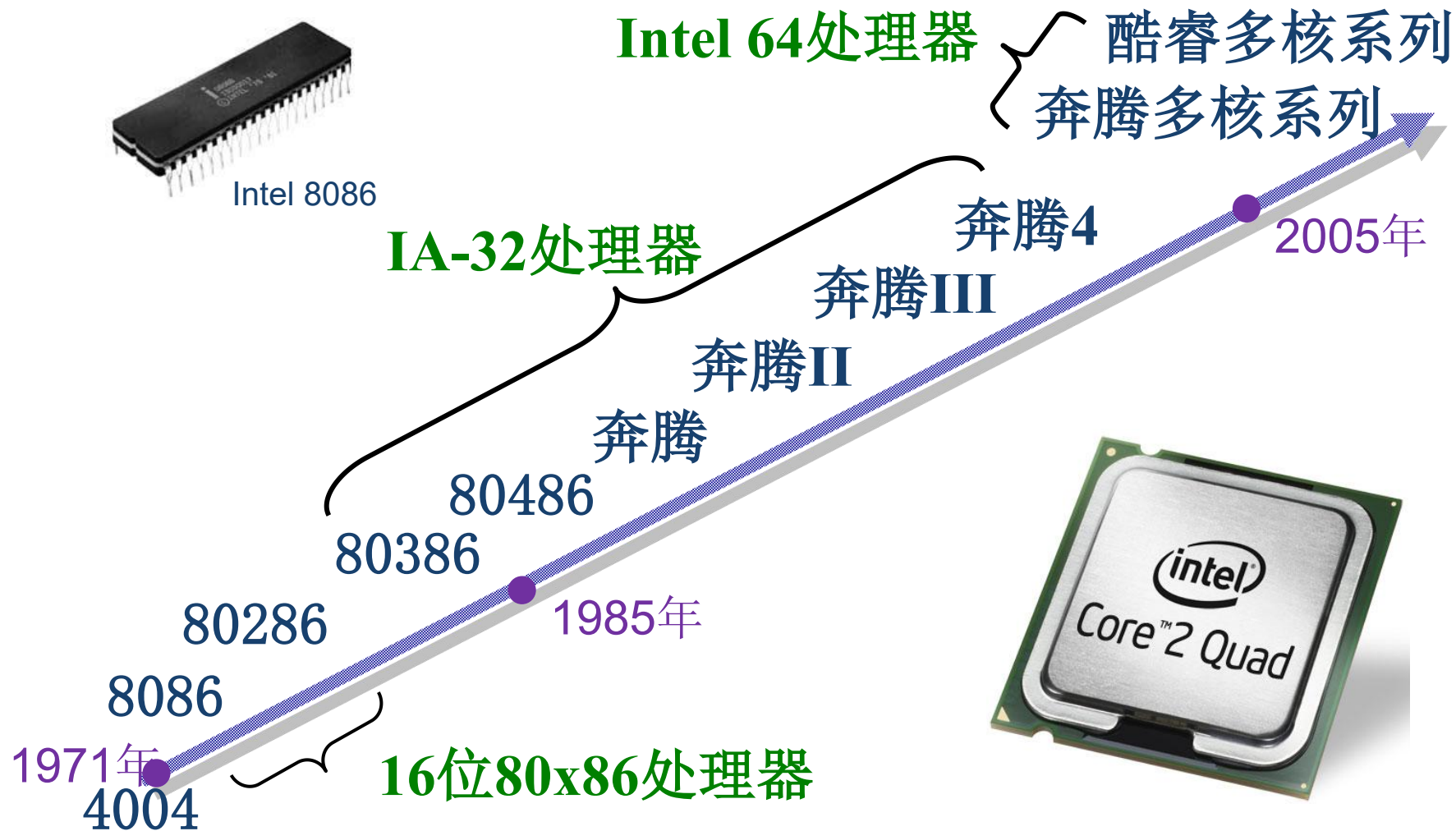
计算机类2024级9班10班

硬件组成



汇编语言程序员将硬件抽象为：
寄存器、存储器地址和输入输出地址

Intel 80x86处理器



x86处理器工作方式

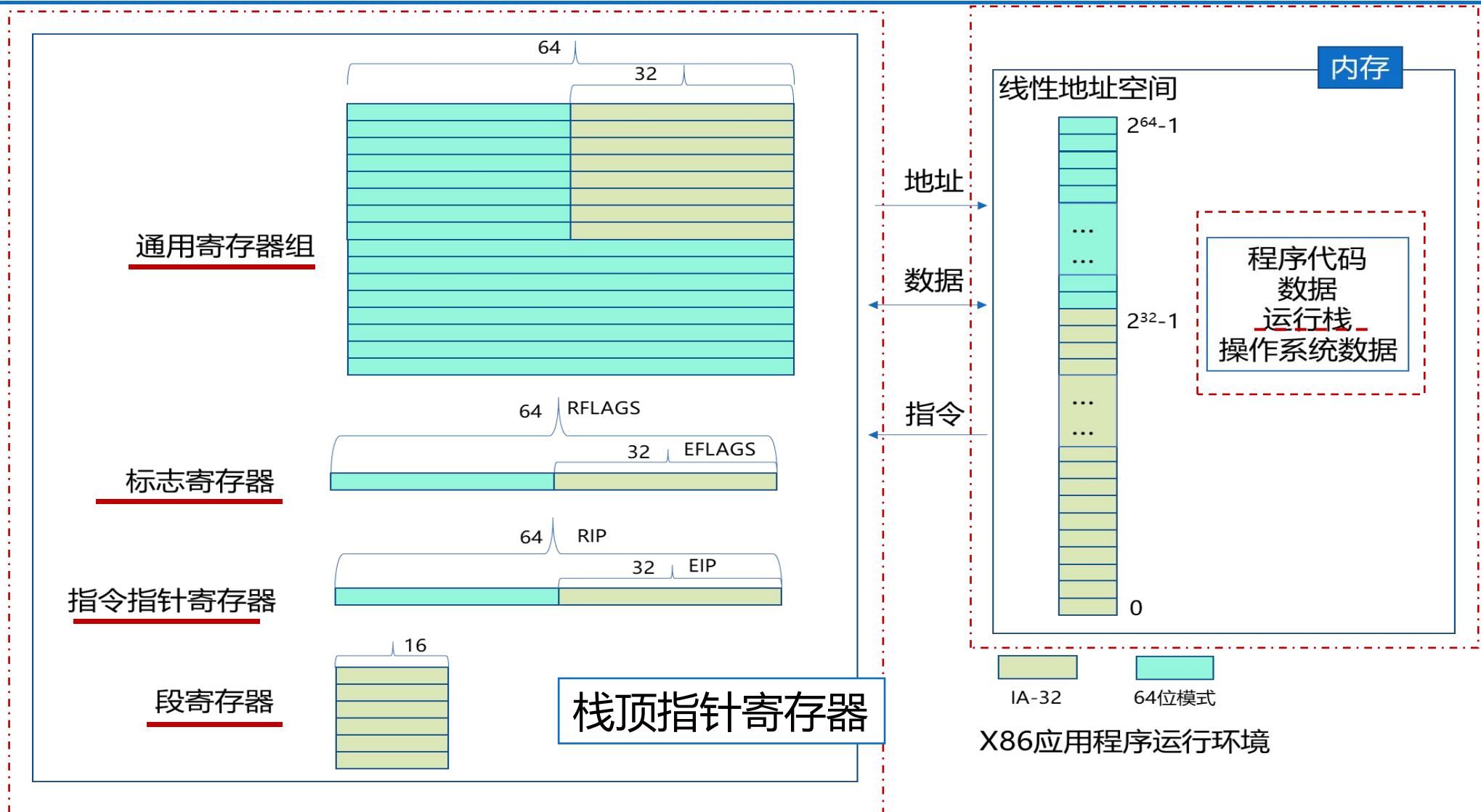
- (1) 保护方式 (Protected Mode)
 - IA-32处理器固有的工作状态
 - 具有强大的段页式存储管理和特权与保护能力
 - 使用全部32条地址总线，可寻址4GB物理存储器
 - 使用平展或段式存储模型
 - 利用虚拟8086方式支持实地址8086软件
- (2) 实地址方式 (Real-address Mode)
 - 可以进行32位处理的快速8086
 - 只能寻址1MB物理存储器空间，每个段不超过64KB
 - 可以使用32位寄存器、32位操作数和32位寻址方式
 - 只能支持实地址存储模型
- (3) 系统管理方式 (System Management Mode)
 - 提供节能和系统安全管理
- (4) IA-32e模式
 - 兼容模式
 - 64位模式

IA-32e模式



- 兼容模式
 - 类似于32位保护模式
 - 应用程序只能访问线性地址空间的前4G字节。
 - 可以使用16位和32位的地址和操作数。
 - 允许大多数传统的16位和32位应用程序在64位操作系统下无需重新编译即可运行
 - 支持64位模式和保护方式中支持的所有特权级别
 - 在虚拟8086模式下运行或适用硬件任务管理的应用程序无法在此模式下工作
- 64位模式
 - 可以访问64位线性地址空间
 - 通用寄存器和SIMD扩展寄存器的数量增加到16个。
 - 通用寄存器位宽扩展到64位

x86运行环境



寄存器 (Register)



- 处理器内部的高速存储单元
- 用于暂时存放程序执行过程中的代码和数据

➤ 透明寄存器

- 对应用人员不可见、不能编程直接控制

➤ 可编程 (Programmable) 寄存器

- 具有引用名称、供编程使用
- 通用寄存器 (General-Purpose Register)
 - 具有多种用途
 - 数量较多、使用频度较高
- 专用寄存器
 - 各自只用于特定目的

通用寄存器

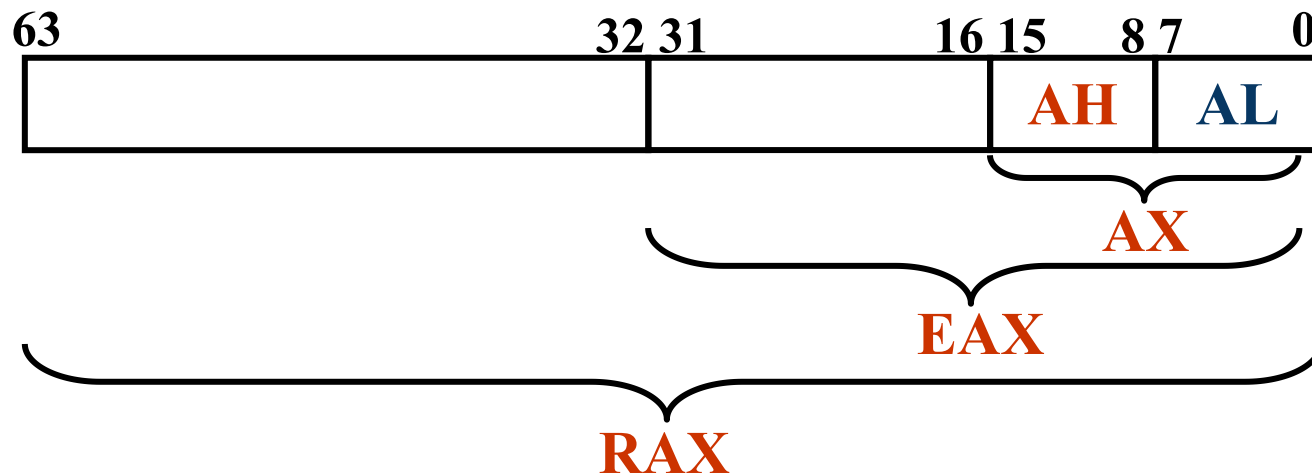
- 处理器最常使用的整数通用寄存器，可用于保存整数数据、地址等
- 16个64位通用寄存器
 - RAX, RBX, RCX, RDX, RSI, RDI, RBP, RSP
 - R8, R9, R10, R11, R12, R13, R14, R15
- 16个32位通用寄存器
 - EAX, EBX, ECX, EDX, ESI, EDI, EBP, ESP
 - R8D, R9D, R10D, R11D, R12D, R13D, R14D, R15D
- 16个16位通用寄存器
 - AX, BX, CX, DX, SI, DI, BP, SP
 - R8W, R9W, R10W, R11W, R12W, R13W, R14W, R15W
- 20个8位通用寄存器
 - AH, BH, CH, DH, AL, BL, CL, DL, SIL, DIL, BPL, SPL
 - R8B, R9B, R10B, R11B, R12B, R13B, R14B, R15B

既是一个整体
又可独立使用

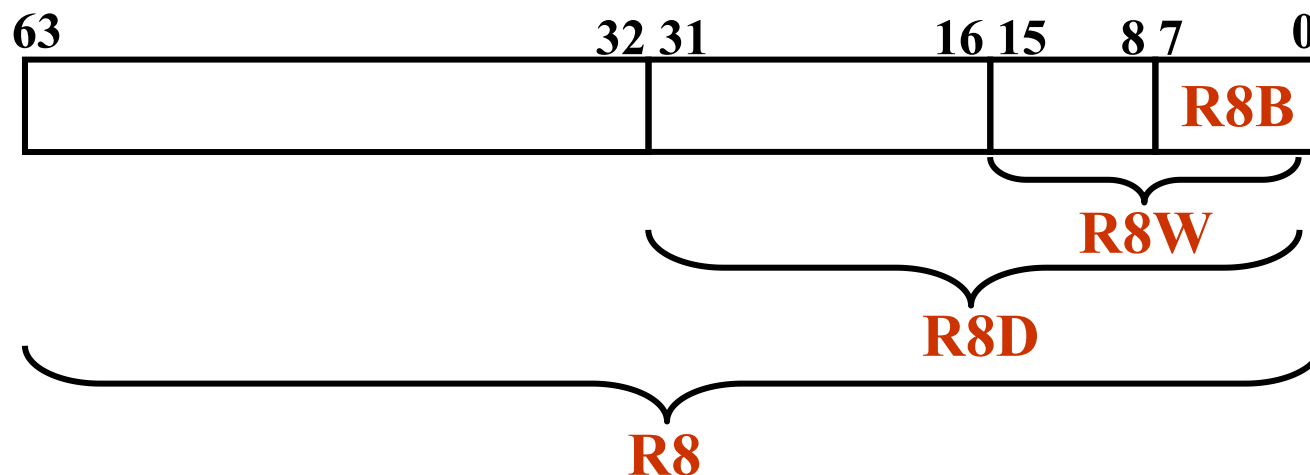
寄存器既是一个整体又可独立使用



扩展



新增



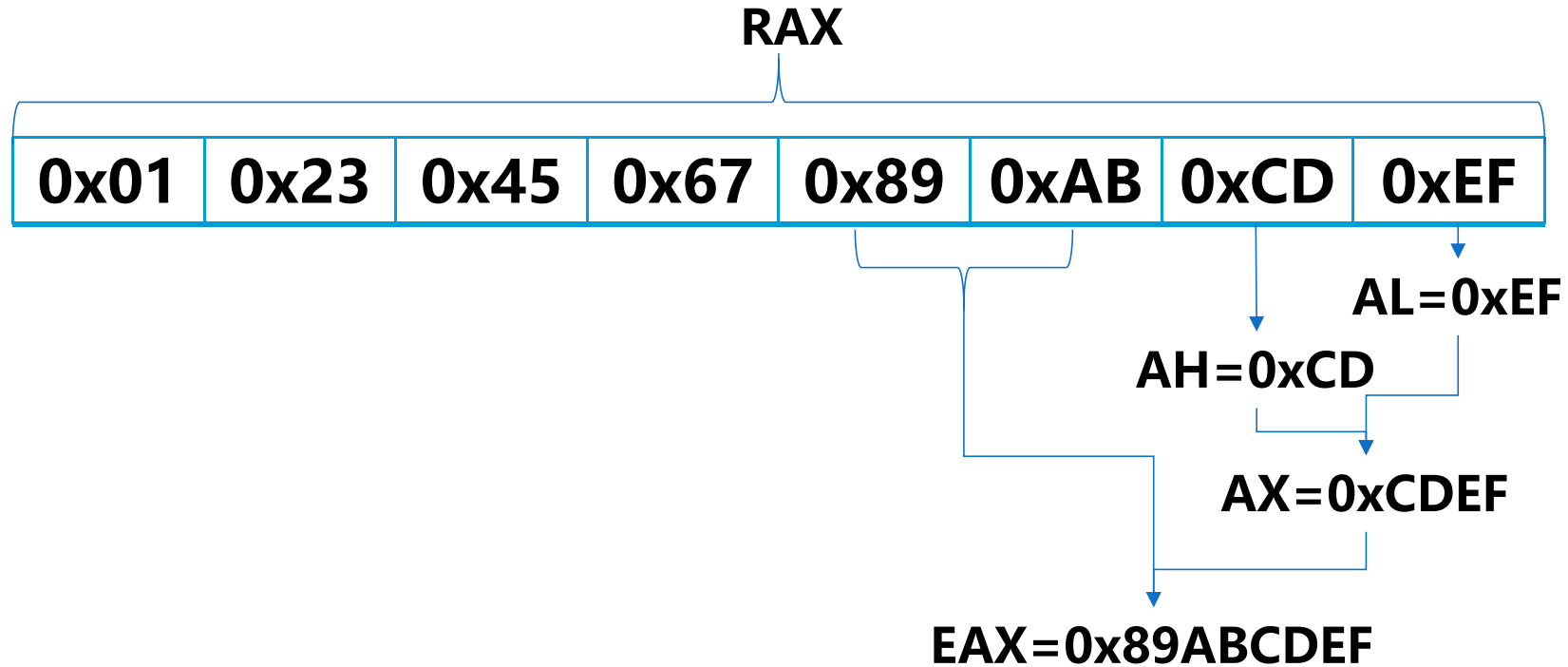
通用寄存器之间的关系

D ₆₃	D ₃₂ D ₃₁	D ₁₆ D ₁₅	D ₈ D ₇	D ₀	D ₀ -D ₁₅	D ₀ -D ₃₁
	RAX	AH	AL		AX	EAX
	RBX	BH	BL		BX	EBX
	RCX	CH	CL		CX	ECX
	RDX	DH	DL		DX	EDX
	RSI		SIL		SI	ESI
	RDI		DIL		DI	EDI
	RSP		SPL		SP	ESP
	RBP		BPL		BP	EBP
	R8		R8B		R8W	R8D
	R9		R9B		R9W	R9D
	R10		R10B		R10W	R10D
	R11		R11B		R11W	R11D
	R12		R12B		R12W	R12D
	R13		R13B		R13W	R13D
	R14		R14B		R14W	R14D
	R15		R15B		R15W	R15D

x86-64处理器:

- 16个64位通用寄存器
- 16个32位通用寄存器
- 16个16位通用寄存器
- 20个8位通用寄存器

RAX=0x123456789ABCDEF



64位模式下字节寄存器访问限制

- 一条指令不能同时引用传统的字节寄存器和新的字节寄存器
- 一条指令可以同时引用传统的高字节和低字节寄存器
 - 传统的高字节寄存器
 - AH、BH、CH、DH
 - 传统的低字节寄存器
 - AL、BL、CL或DL
 - 新的字节寄存器
 - R8B~R15B, DIL, SIL, BPL, SPL

例如:

<code>mov ah, al</code>	✓
<code>mov ah,R8B</code>	✗
<code>mov al,dil</code>	✗

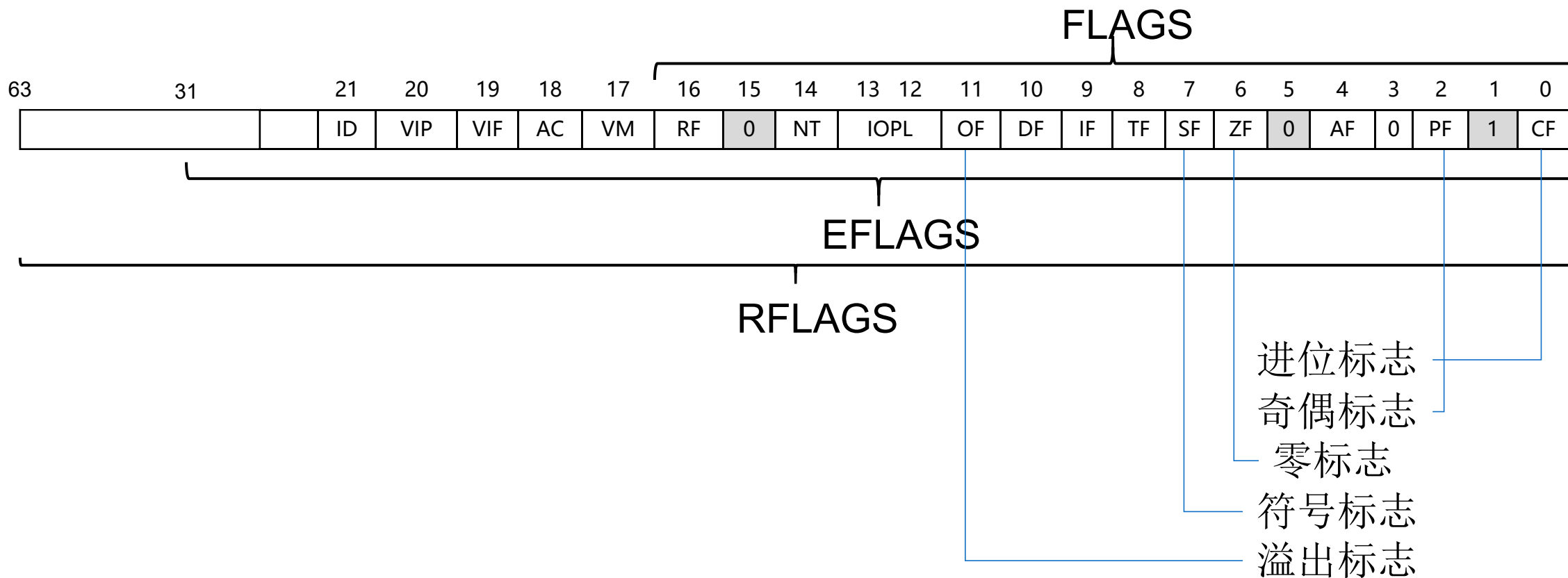
随堂练习：寄存器

处理器专用寄存器



标志寄存器	RFLAGS
指令指针寄存器	RIP
段寄存器	CS DS SS
	ES FS GS

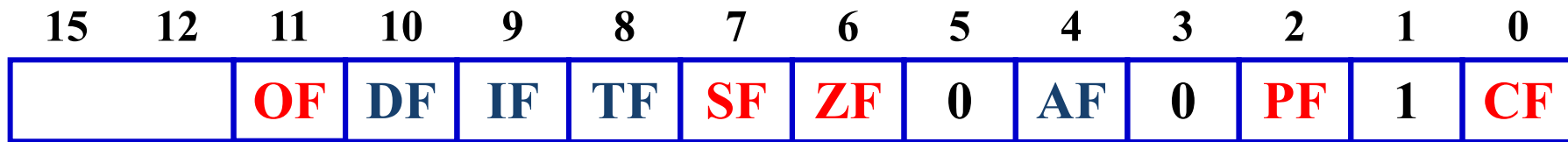
标志寄存器



随堂练习：标志位设置

处理器最基本的标志：状态标志

- 用来记录指令执行结果的辅助信息
- 加减运算和逻辑运算指令主要设置它们
- 其他有些指令的执行也会相应地设置它们
- 处理器主要使用其中5个构成各种条件，分支指令判断这些条件实现程序分支



8086的标志

进位标志CF (Carry Flag)

- 当加减运算结果的最高有效位有进位（加法）或借位（减法）时，进位标志置1，即 $CF = 1$ ；否则 $CF = 0$
- 针对无符号整数，判断加减结果是否超出表达范围
 - N个二进制位表达无符号整数的范围： $0 \sim 2^N - 1$
 - 8位： $0 \sim +255$
 - 16位： $0 \sim +65535$
 - 32位： $0 \sim +2^{32} - 1$
 - 64位： $0 \sim +2^{64} - 1$

进位标志CF：举例

- 8位二进制数相加：

$$\underline{00111010 + 01111100 = 10110110}$$

- 十六进制表达： $3A + 7C = B6$

- 转换成十进制数： $58 + 124 = 182$

- 没有产生进位： $CF = 0$

$$0 < 182 < 255$$

- 8位二进制数相加：

$$\underline{10101010 + 01111100 = [1]00100110}$$

- 十六进制表达： $AA + 7C = [1]26$

- 转换成十进制数： $170 + 124 = 294 = 256 + 38$

- 产生进位： $CF = 1$

$$\text{进位1表达256}$$

溢出标志OF (Overflow Flag)

- 有符号数加减结果有溢出，则OF = 1；否则OF = 0
- 针对有符号整数，判断加减结果是否超出表达范围
 - N个二进制位（补码）表达有符号整数的范围： $-2^{N-1} \sim 2^{N-1}-1$
 - 8位： $-128 \sim +127$
 - 16位： $-32768 \sim +32767$
 - 32位： $-2^{31} \sim +2^{31} - 1$
 - 64位： $-2^{63} \sim +2^{63} - 1$

杯中水已满，
再加就溢出！

溢出标志OF：举例

- 8位二进制数相加：

$$\underline{00111010 + 01111100 = 10110110}$$

- 十六进制表达： $3A + 7C = B6$

- 转换成十进制数： $\underline{58 + 124 = 182}$

- 超出范围： $\underline{OF = 1}$

$$182 > 127$$

- 8位二进制数相加：

$$\underline{10101010 + 01111100 = [1]00100110}$$

- 十六进制表达： $AA + 7C = [1]26$

- 转换成十进制数： $\underline{-86 + 124 = 38}$

- 没有超出范围： $\underline{OF = 0}$

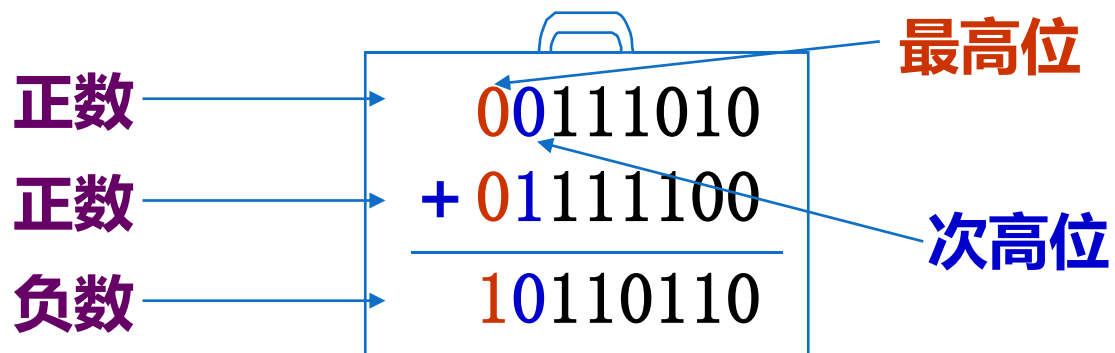
$$\text{补码 } 0xAA \text{ 表达 } -86$$

进位和溢出的区别

- 进位标志反映无符号整数运算结果是否超出范围
 - 有进位，加上进位或借位后运算结果仍然正确
- 溢出标志反映有符号整数运算结果是否超出范围
 - 有溢出，运算结果已经不正确
- 处理器按照无符号整数求得结果
 - 设置进位标志CF
 - 设置溢出标志OF
- 程序员决定
 - 操作数是无符号数，关心进位
 - 操作数是有符号数，注意溢出

溢出标志的判断

- 处理器硬件判断规则
 - 最高位和次高位同时有进位或同时无进位，无溢出；最高位和次高位进位状态不同，有溢出
- 人工判断的简单规则
 - 只有当两个相同符号数相加（含两个不同符号数相减），而运算结果的符号与原数据符号相反时，产生溢出；其他情况下，不会产生溢出



零标志ZF (Zero Flag)

- 运算结果为0，则ZF = 1，否则ZF = 0

结果是0，
ZF标志不是0！

举例

- 8位二进制数相加：

$$00111010 + 01111100 = 10110110$$

结果不是0，ZF = 0

- 8位二进制数相加：

$$10000100 + 01111100 = [1]00000000$$

结果是0，ZF = 1

进位

结果

符号标志SF (Sign Flag)

- 运算结果最高位为1, 则SF = 1; 否则SF = 0

最高位 = 符号位 = SF

举例

- 8位二进制数相加:

$$00111010 + 01111100 = 10110110$$

最高位 = 1: SF = 1

- 8位二进制数相加:

$$10000100 + 01111100 = [1]00000000$$

最高位 = 0: SF = 0

进位

结果

奇偶标志PF (Parity Flag)

- 当运算结果最低字节中“1”的个数为零或偶数时，PF = 1；
否则PF = 0

仅最低8位“1”的个数

举例

- 8位二进制数相加：

$$00111010 + 01111100 = 10110110$$

“1”的个数为5个：PF=0

- 8位二进制数相加：

$$10000100 + 01111100 = [1]00000000$$

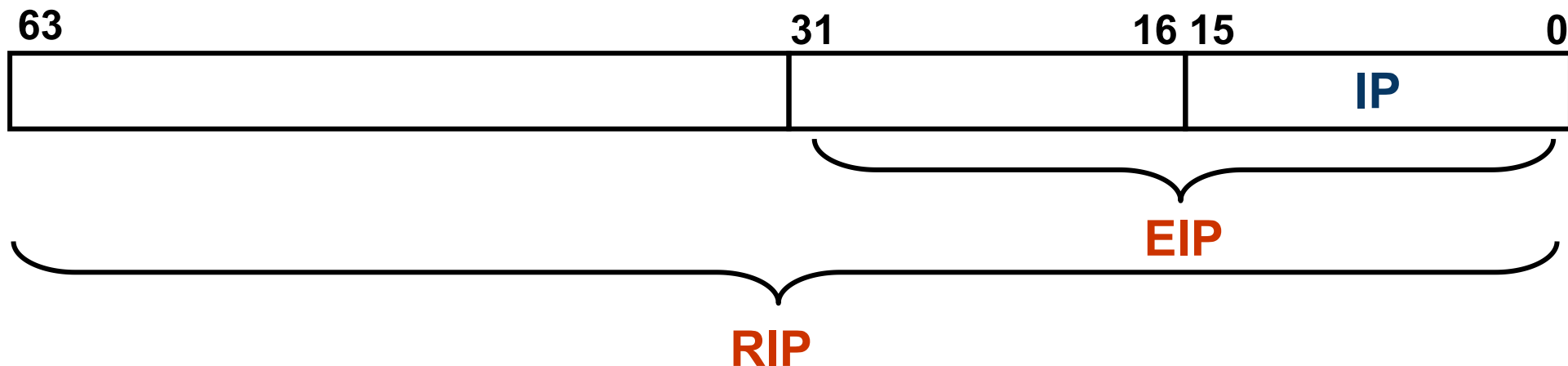
“1”的个数为0个：PF=1

进位

结果

指令指针寄存器RIP

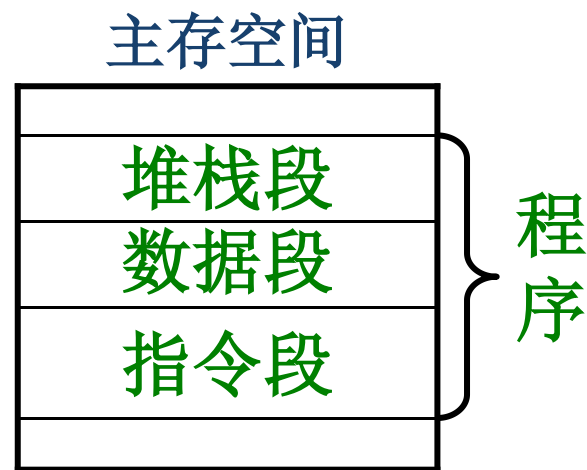
- 保存将要执行的指令在主存的存储器地址
- RIP是专用寄存器
 - 顺序执行时自动增量（加上该指令的字节数），指向下一条指令
 - 分支、调用等操作时需执行控制转移指令修改，引起程序转移到指定的指令执行
 - 出现中断或异常时被处理器赋值而相应改变



段寄存器



- 段 (Segment) 是用于安排相关代码或数据的一个主存区域
- 段寄存器表明某个段在主存中的位置
- 6个16位段寄存器: CS DS SS ES FS GS
- 应用程序主要涉及3类段
 - 代码段 (Code Segment)
存放程序中指令代码
 - 数据段 (Data Segment)
存放当前运行程序所用数据
 - 堆栈段 (Stack Segment)
指明程序使用的堆栈区域



IA-32处理器按逻辑段组织程序，数据变量应该且必须在数据段中。 ❌

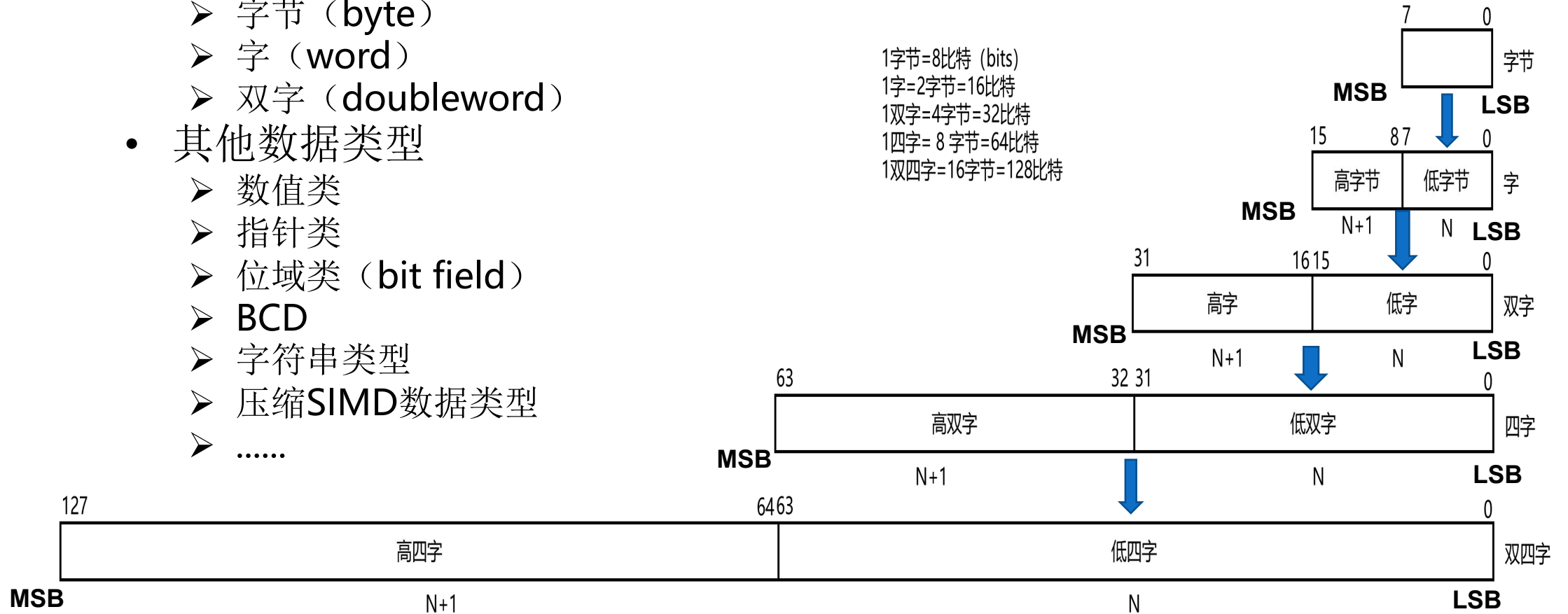
IA-32处理器读取指令，一定是在代码段中。 ✔️

x86处理器支持的数据类型



- 基本数据类型
 - 字节 (byte)
 - 字 (word)
 - 双字 (doubleword)
- 其他数据类型
 - 数值类
 - 指针类
 - 位域类 (bit field)
 - BCD
 - 字符串类型
 - 压缩SIMD数据类型
 -

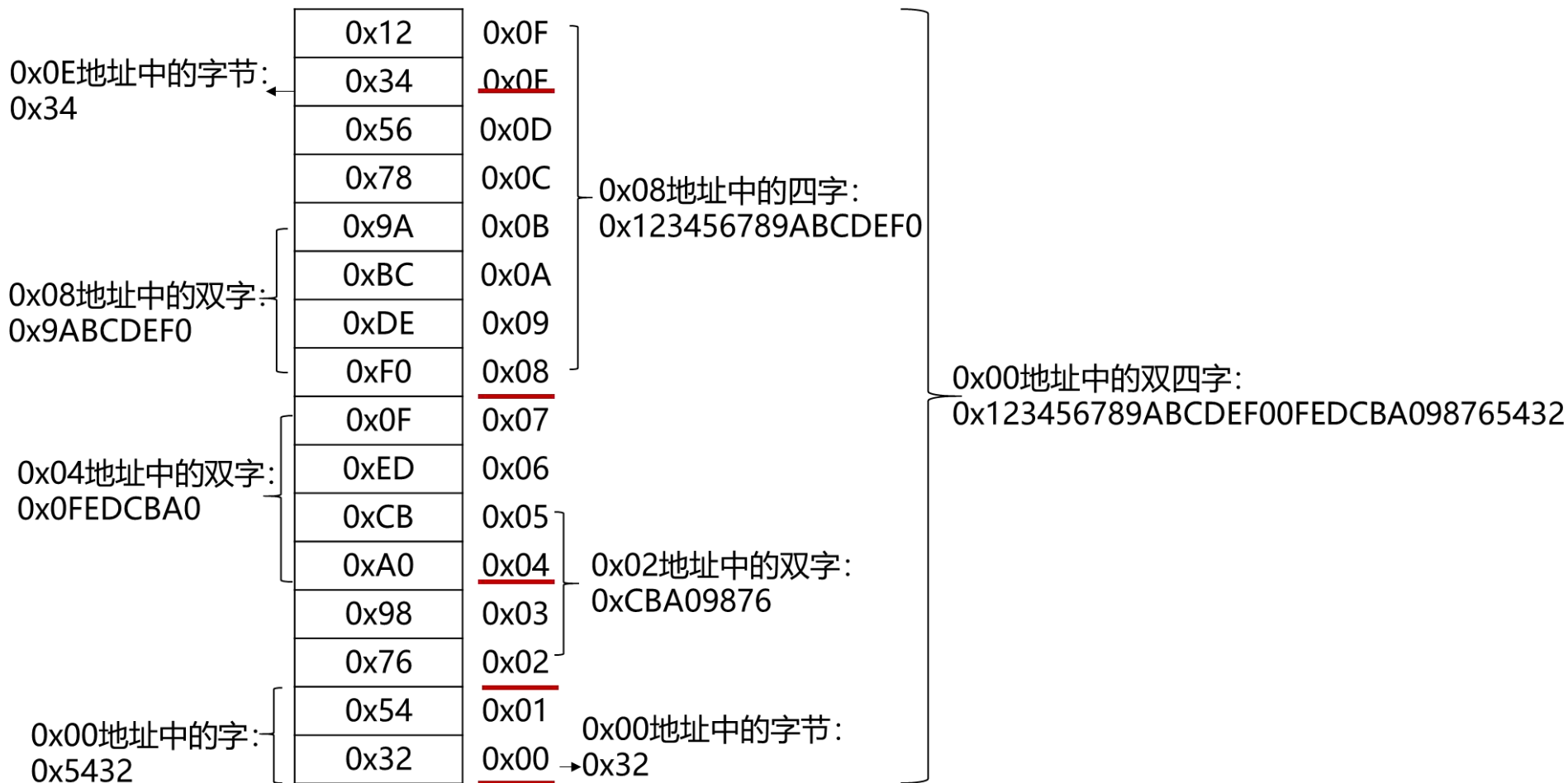
1字节=8比特 (bits)
1字=2字节=16比特
1双字=4字节=32比特
1四字= 8 字节=64比特
1双四字=16字节=128比特



小端方式存储



- 低字节在低地址，高字节在高地址



- 物理存储器以字节为基本存储单位
- 物理地址
 - 每个存储单元被分配一个唯一的地址
- 物理地址空间从0开始顺序编排，直到处理器支持的最大存储单元
- 8086处理器支持1MB存储器：0x00000 ~ 0xFFFFF
- IA-32处理器支持4GB存储器：0x00000000 ~ 0xFFFFFFFF
- x86-64处理器支持4GB存储器：0x00000000 ~ 0xFFFFFFFF
- 操作系统利用存储管理单元进行存储管理，程序不直接寻址物理存储器
- x86处理器提供3种存储模型，用于程序访问存储器

x86处理器的存储模型

(1) 平展存储模型 (Flat Memory Model)

- 存储器是一个连续的地址空间：线性地址空间
- IA-32处理器支持4GB容量线性地址空间

(2) 段式存储模型 (Segmented Memory Model)

- 存储器由一组独立的地址空间：段 (Segment)
- IA-32处理器每个段都可以达到4GB容量
- 在处理器内部，所有的段都被映射到线性地址空间

(3) 实地址存储模型 (Real-address Memory Model)

- 8086处理器的存储模型
- 段式存储模型的特例
- 线性地址空间最大为1MB容量，段最大为64KB

工作方式与存储模型



工作方式

存储模式

保护方式
(虚拟8086方式)

系统管理方式

实地址方式

兼容模式

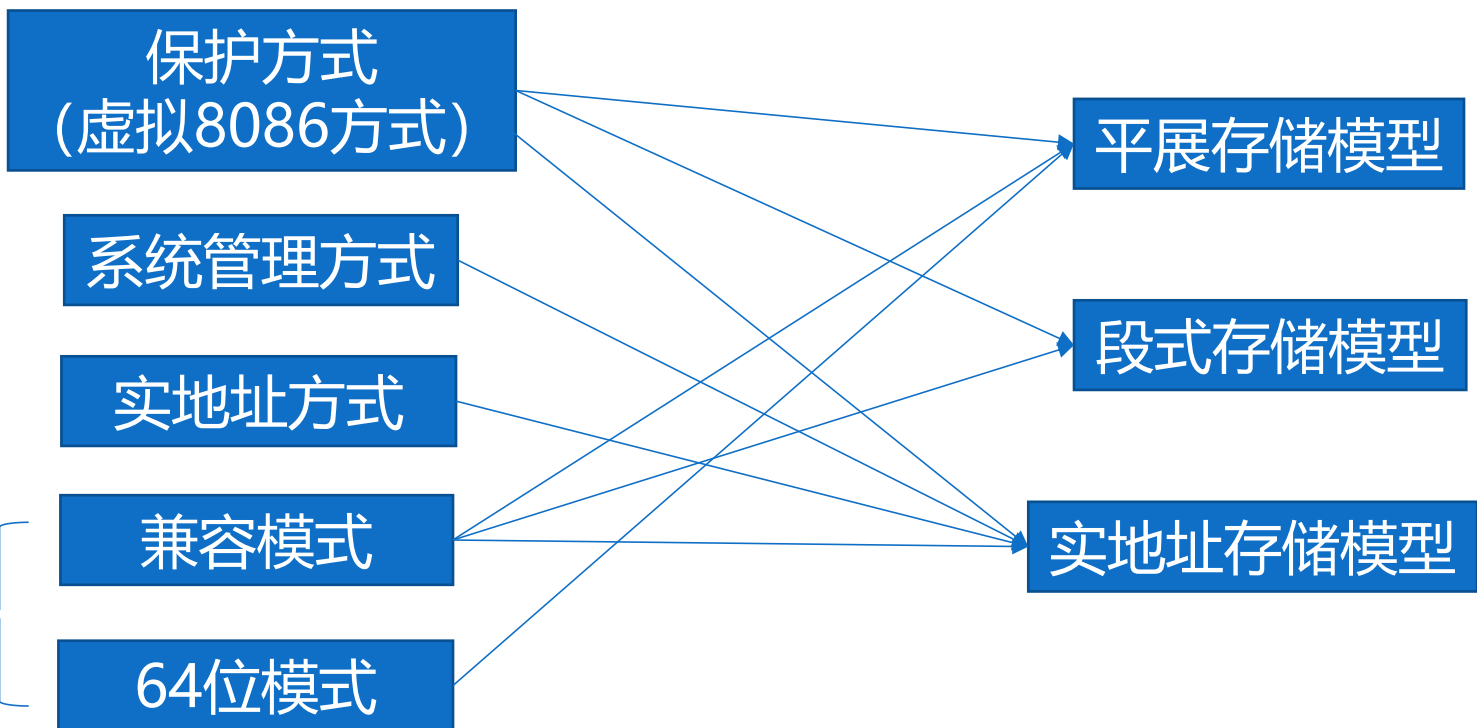
64位模式

平展存储模型

段式存储模型

实地址存储模型

IA-32e模式



物理地址与逻辑地址



- 物理地址
 - 物理存储器以字节为基本存储单位，每个存储单元被分配一个唯一的地址
- 程序中使用逻辑地址
 - 在处理器内部、程序员编程时采用的地址
 - 逻辑地址 = 段基地址 : 偏移地址
 - 段基地址 = 在主存中的起始地址
 - 偏移地址 = 距离段基地址的位移量
- 某个存储单元可以有多个逻辑地址，但只有一个唯一的物理地址

绝对地址与相对地址



逻辑地址 = 相对地址: **205** (2层05号房间)

301	302	303	304	305	306	307	308	309	310
201	202	203	204	205	206	207	208	209	210
101	102	103	104	105	106	107	108	109	110

物理地址 = 绝对地址: **15** (第15号房间)

21	22	23	24	25	26	27	28	29	30
11	12	13	14	15	16	17	18	19	20
01	02	03	04	05	06	07	08	09	10

基本段的逻辑地址

- 代码段 (Code Segment)

- 段基地址：代码段寄存器CS指示
- 偏移地址：指令指针寄存器EIP保存

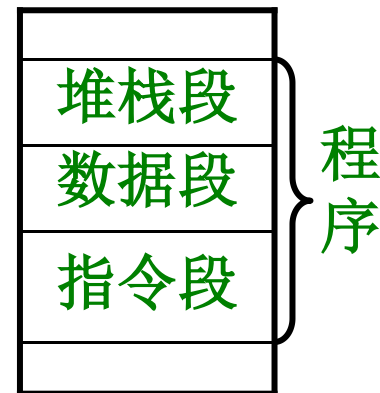
- 数据段 (Data Segment)

- 段基地址：数据段寄存器DS指示
 - 有时也用附加段寄存器ES，段寄存器FS和GS指示
- 偏移地址：各种存储器寻址方式计算出来

- 堆栈段 (Stack Segment)

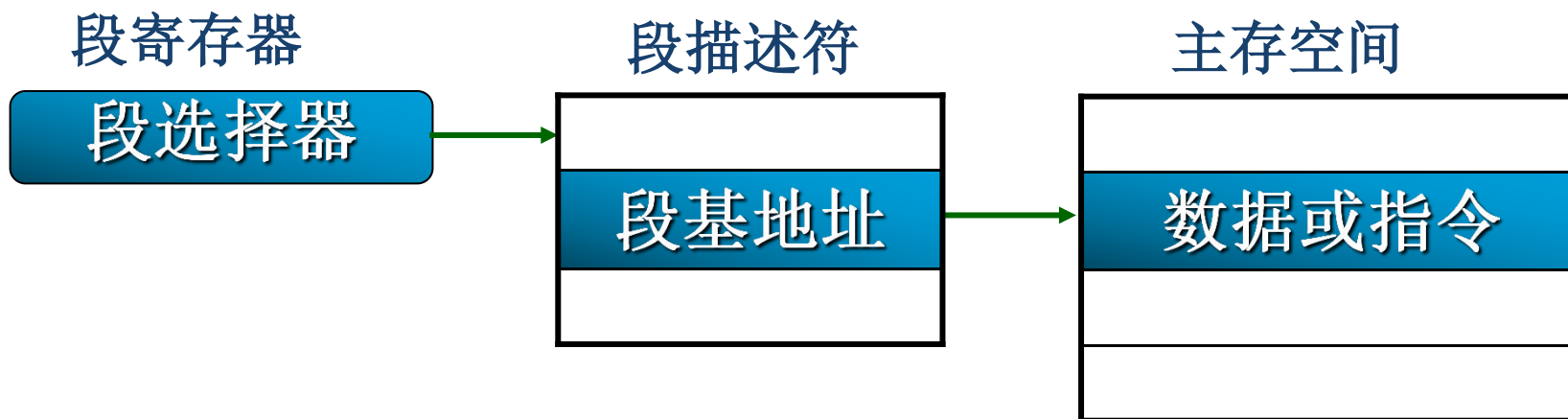
- 段基地址：堆栈段寄存器SS指示
- 偏移地址：堆栈指针寄存器ESP保存

主存空间



段选择器

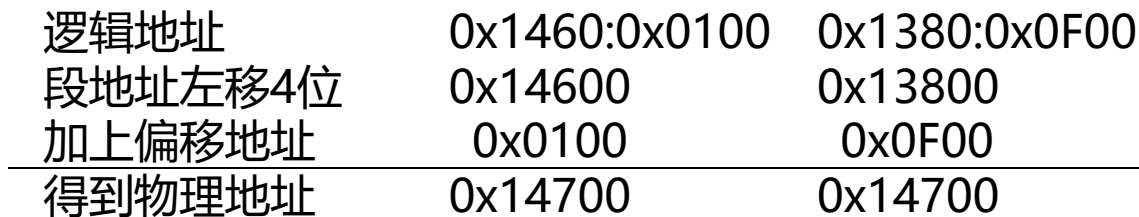
- 16位段寄存器保存16位段选择器
- 段选择器指向64位段描述符 (Descriptor)
- 段描述符包括段基地址
 - 平展存储模型：指向地址0位置
 - 段式存储模型：指向线性地址空间不同的段
 - 实地址存储模型：保存段基地址的高16位



练习



- 逻辑地址0x1460:0x0100对应的物理地址=?





操作数寻址方式

数据寻址方式

- 指令有两部分：操作码和操作数
 - 操作码：处理器要执行哪种操作
不可缺少，用助记符表示
 - 操作数：指令执行的参与者
各种操作的对象，需要通过地址指示
- 数据寻址方式：通过地址查找数据（操作数）
 - 立即数寻址：数据在指令代码中，用常量表达
 - 寄存器寻址：数据在寄存器中，用寄存器名表示
 - 存储器寻址：数据在主存中，用存储器地址代表

使用最多、最基本的数据传送指令

- 传送指令的助记符：MOV（取自Move）
 - 将数据从一个位置传送到另一个位置
 - 类似高级语言的赋值语句

`mov dest,src`

;源操作数src：被传送的数据或数据所在的位置

;目的操作数dest：数据将要传送到的位置

`mov eax, ebx`

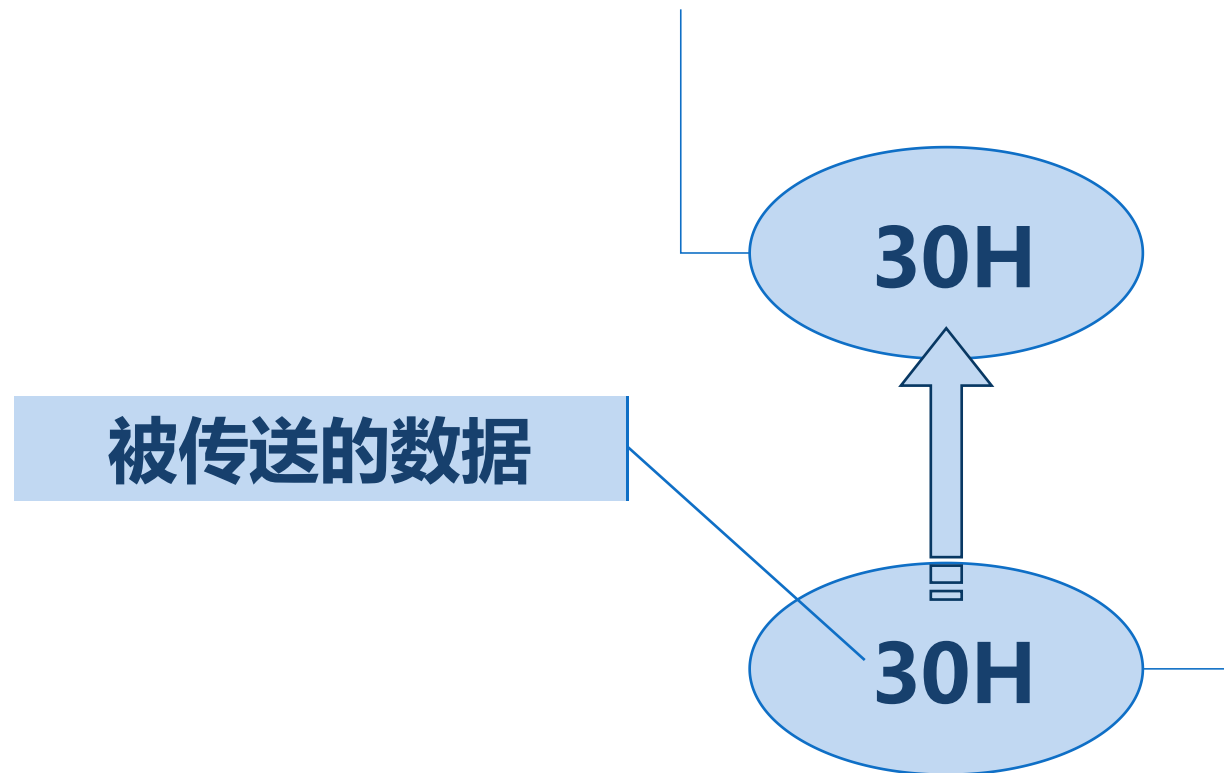
`mov rax, [rbx]`

`mov edx, [rbx+rsi*4+0x80]`

MOV指令的功能

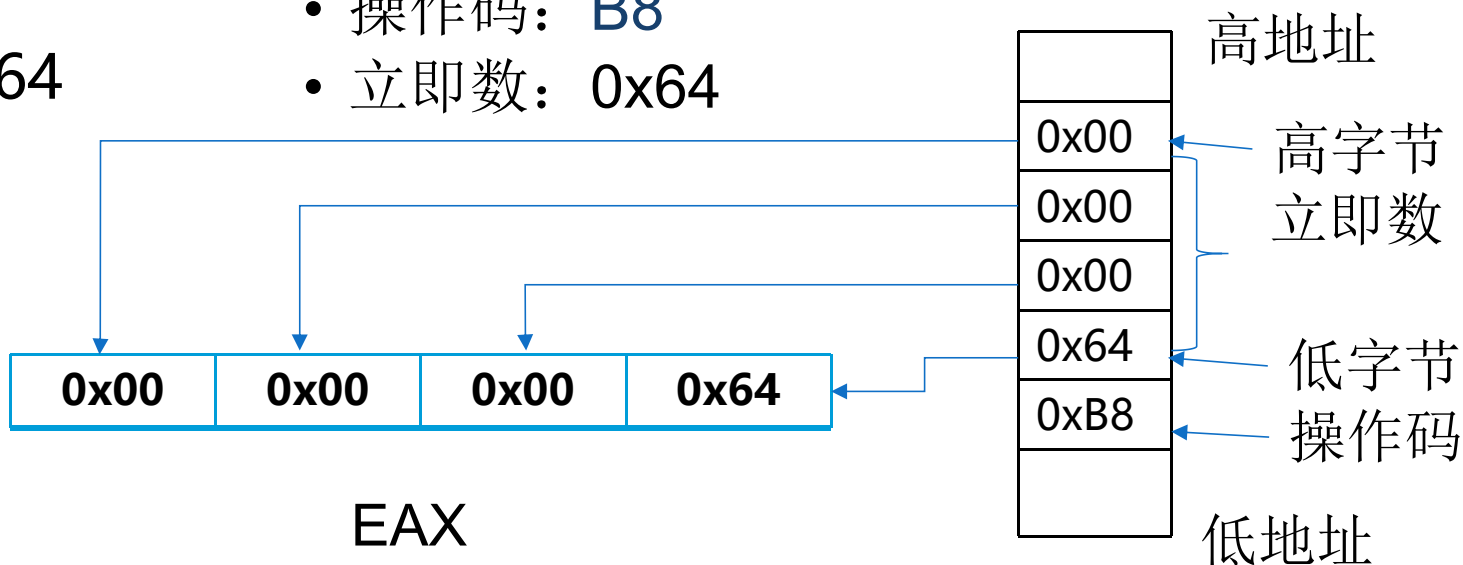


mov 目的操作数 ， 源操作数



立即数寻址

- 操作数紧跟操作码，是机器代码的一部分
- 操作数从指令代码中立即得到，即立即数 (Immediate) ，用常量形式直接表达
- 立即数寻址方式只用于源操作数，常用来给寄存器和存储单元赋值
- 例如：**mov eax, 100**
 - 机器代码：B8 64 00 00 00
 - 操作码：B8
 - 立即数：0x00000064
- **mov al, 100**
 - 机器代码：B8 64
 - 操作码：B8
 - 立即数：0x64



例3-1



指令

机器码

mov al, 100

B0	64
----	----

mov ax, 100

66	B8	64	00
----	----	----	----

mov eax, 100

B8	64	00	00	00
----	----	----	----	----

mov rax, 100

48	C7	C0	64	00	00	00
----	----	----	----	----	----	----

寄存器寻址

- 操作数存放在处理器的内部寄存器中
- 用寄存器名表示它的内容
- 绝大多数指令采用通用寄存器寻址
- 部分指令支持专用寄存器，例如段寄存器
- 寄存器寻址方式简单快捷，最常使用
- 例如：MOV EBX,EAX

64位通用寄存器：RAX RBX RCX RDX

32位通用寄存器：EAX EBX ECX EDX

16位通用寄存器：AX BX CX DX

8位通用寄存器：AH AL BH BL

寄存器寻址示例



指令

机器码

mov ax, bx

66	89	D8
----	----	----

mov dl,al

88	C2
----	----

mov ch, cl

88	C2
----	----

mov esi, edx

89	D6
----	----

mov rax, r8

4C	89	C0
----	----	----

mov edi, si ❌

instruction operands must be the same size

- **操作数在主存中，通过存储器地址指示**
- 编程时，存储器地址使用包含段选择器和偏移地址的逻辑地址
 - 段选择器（段寄存器）指示段基地址
 - 默认规定：数据在DS指向的数据段；EBP或ESP作为基地址，数据在SS指向的堆栈段
 - 显式说明：使用段超越指令前缀，段寄存器名后跟英文冒号
 - 偏移地址由各种寻址方式计算
 - 常被称为**有效地址EA**（Effective Address）

偏移地址的组成

64位有效地址 = 基址寄存器 + (变址寄存器 × 比例) + 位移量

- 基址寄存器：任何一个64位通用寄存器之一，RIP
- 变址寄存器：除RSP外的任何64位通用寄存器之一
- 比例：1, 2, 4, 8或16
- 位移量：8, 16, 32或64位有符号值



变化出多种主存寻址方式

1. 直接寻址

- 有效地址只有位移量部分，直接包含在指令代码中
- 常用于存取变量
- 仅适用于IA-32处理器保护模式
- 64位模式下不使用这种寻址方式
- 例如：在IA-32处理保护模式下
`MOV ECX, [COUNT]` #COUNT是变量

`MOV ECX, DS:[405000H]`

- 指令代码：8B 0D 00 50 40 00
- 操作码和寻址方式：8B 0D
- 操作数：有效地址 00405000H

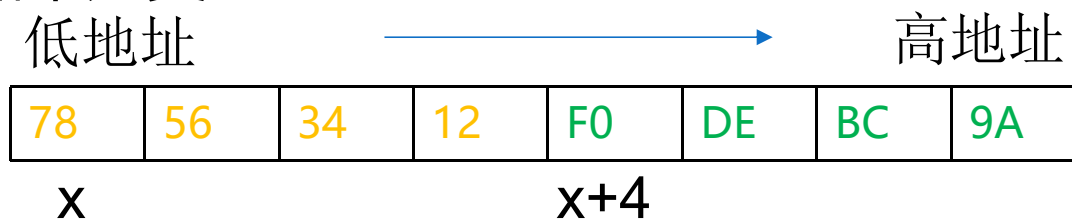
2.RIP相对寻址方式

- 64位模式下最常使用的一种存储器寻址方式
- 有效地址 = RIP+64位位移量

行数	线性地址	机器码	汇编语句
			.data
9	0000	78563412	x: .long 0x12345678,0x9ABCDEF0
9		F0DEBC9A	
10	0008	31323334	y: .ascii "12345"
10		35	
			.text
14	0000	8B050000	mov eax,x[rip] /*有效地址=x+rip,eax=0x12345678 */
14		0000	
17	0010	8B050800	mov eax,y[rip] /*有效地址=y+rip,eax=0x34333231 */
17		0000	
20	0020	488B0504	mov rax,x[rip+4] /*有效地址=x+rip+4,rax=343332319ABCDEF0 */
20		000000	

3. 寄存器间接寻址

- 有效地址存放在寄存器中(寄存器内容 = 偏移地址 = 有效地址)
- 用中括号括起寄存器
- 可以方便地对数组的元素或字符串的字符进行操作
- 寄存器间接寻址没有说明存储单元类型
- 例如:



```
.data
x: .long 0x12345678,0x9ABCDEF0
.text
lea rax, x[rip] /*x地址复制到rax*/
mov bl, [rax] /*有效地址=rax, bl=0x78 */
mov bx, [rax] /*有效地址=rax, bx=0x5678 */
mov ebx,[rax] /*有效地址=rax, ebx=0x12345678 */
mov rbx,[rax] /*有效地址=rax, rax=0x9ABCDEF012345678 */
```

4. 寄存器相对寻址

- 有效地址是寄存器内容与位移量之和
- 可以方便地对数组的元素或字符串的字符进行操作
- 例如:

```
.data  
x: .long 0x12345678, 0x9ABCDEF0
```

```
.text
```

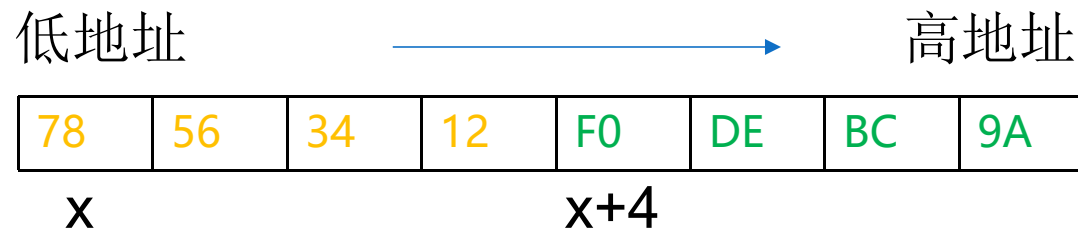
```
lea rax, x[rip] /*x地址复制到rax*/
```

```
mov bl, [rax+1] /*有效地址=rax+1, bl=0x56 */
```

```
mov bx, [rax+2] /*有效地址=rax+2, bx=0x1234 */
```

```
mov ebx, [rax+4] /*有效地址=rax+3, ebx =0x9ABCDEF0 */
```

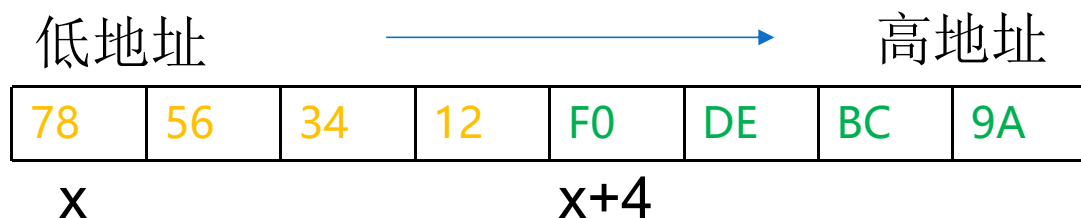
```
mov rbx, [rax+4] /*有效地址=rax+4, rbx =0x000000009ABCDEF0*/
```



主存以字节为可寻址单位
地址的加减是以字节为单位

5. 基址变址寻址

- 使用变址寄存器寻址操作数
- 便于支持二维数组等数据结构
- 例如：

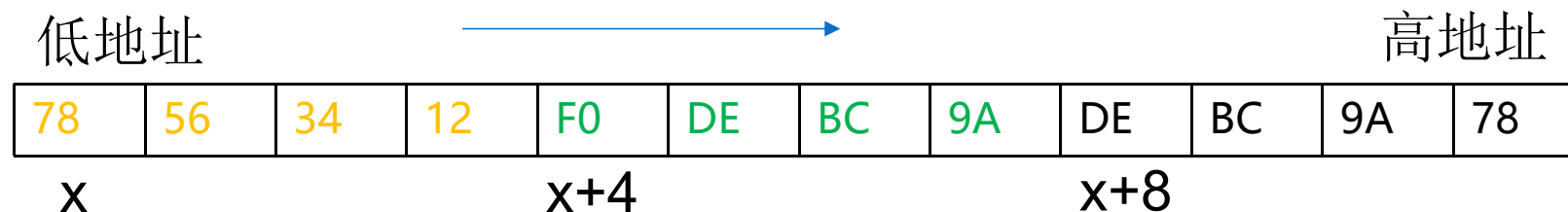


```
.data
x: .long 0x12345678, 0x9ABCDEF0
.text
lea rax,x[rip] /*x地址复制到rax*/
mov rbx, 0
mov ebx,[rax+rbx] /*有效地址=rax+rbx, ebx=0x12345678 */
mov rbx, 4
mov ebx,[rax+rbx] /*有效地址=rax+rbx, ebx=0x9ABCDEF0 */
```

6.相对基址变址寻址

- 有效地址由基址寄存器的内容加上变址寄存器内容再加上一个64位的位移量构成

• **有效地址 = 基址寄存器 + 变址寄存器 + 64位位移量**



.data

x: .long 0x12345678, 0x9ABCDEF0, 0x789ABCDE

.text

lea rax, x[rip] /*x地址复制到rax*/

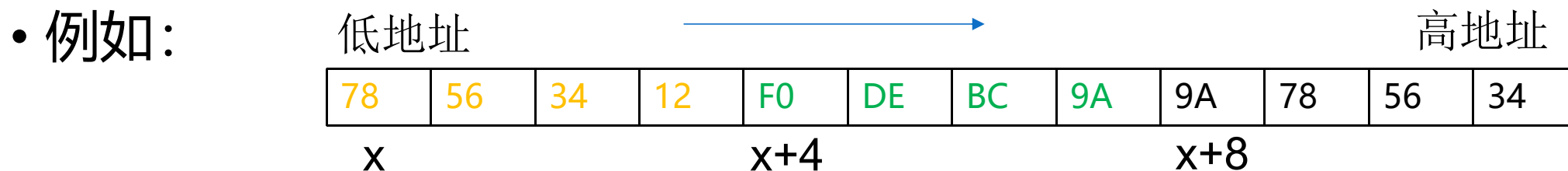
mov rbx, 4

mov eax, [rax+rbx+2] /*有效地址=rax+rbx+2, eax = 0xBCDE9ABC */

mov eax, [rax+rbx+4] /*有效地址=rax+rbx+4, eax = 0x789ABCDE */

7. 带比例的基址变址寻址

- 变址寄存器内容乘以比例1（可省略），2，4或8的变址寻址
- 比例1、2、4和8对应8、16、32和64位数据的字节个数，便于以数组元素为单位寻址相应数据



```
.data
x: .long 0x12345678, 0x9ABCDEF0, 0x3456789A
.text
lea rax,x[rip] /*x地址复制到rax*/
mov rbx, 1
mov eax,[rax+rbx*4] /*有效地址=rax+rbx+2, eax = 0x9ABCDEF0 */
mov eax,[ rax+rbx*4+4] /*有效地址=rax+rbx+4, eax = 0x3456789A */
```

例3-9 数组访问

```
//eg0309.c
#include <stdio.h>
int array[]={12345,-12345};
int i,*pi=array;
int main(){
    printf("%d\n",array[0]);
    i=1;
    printf("%d\n",array[i]);
    i=2;
    printf("%d\n",array[i]);
    printf("%d\n",*pi);
    pi++;
    printf("%d\n",*pi);
    pi++;
    printf("%d\n",*pi);
    return 0;
}
```

mov	edx, DWORD PTR array[rip]	#RIP相对寻址, array[0]
mov	edx, DWORD PTR array[rip+4]	#RIP相对寻址, array[1]
mov	edx, DWORD PTR array[rip+8]	#RIP相对寻址, array[2]
mov	rax, QWORD PTR pi[rip]	/*rax=pi*/
mov	edx, DWORD PTR [rax]	/*寄存器间接寻址, *pi */
mov	rax, QWORD PTR pi[rip]	
lea	rdx, 4[rax]	#rdx=pi+4
mov	QWORD PTR pi[rip], rdx	
mov	edx, DWORD PTR 4[rax]	#寄存器相对寻址, *(pi+1)
mov	rax, QWORD PTR pi[rip]	
lea	rdx, 4[rax]	#rax=pi+8
mov	QWORD PTR pi[rip], rdx	
mov	edx, DWORD PTR 4[rax]	#寄存器相对寻址, *(pi+2)

例3-9 运行与调试



运行结果

```
12345
-12345
1109221296
12345
-12345
1109221296
```

数据泄露!

GDB查看变量地址: **p 变量地址**

```
(gdb) p &i
$1 = 0x7ff7421da080 <i>
```

```
(gdb) p &pi
$2 0x7ff7421d4010 <pi>
```

```
(gdb) p &array
$3 = 0x7ff7421d4018 <array>
```

$\&\text{array}[2] = 0x7ff7421d4020 = 0x7ff7421d4018 + 8$

```
(gdb) x/dw 0x7ff7421d4020
0x7ff7421d4020 <p.0>: 1109221296
```

- ▶ **指令格式**
- ▶ **指令助记符、功能、注意事项**
- ▶ **对标志位的影响**

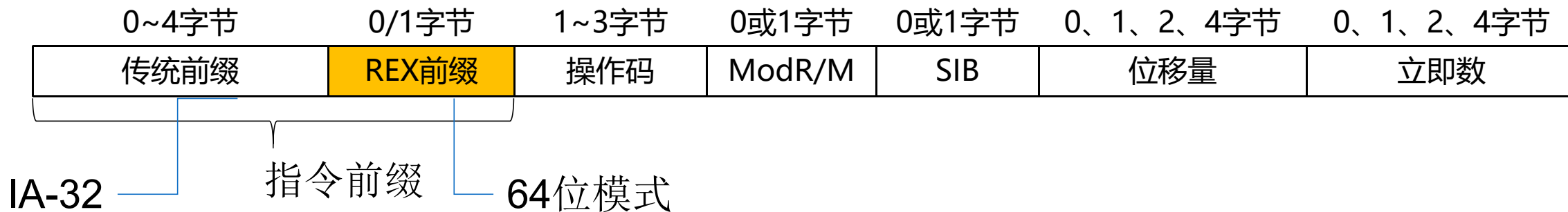
传送和运算类指令导学

教材章节：4.1-4.3

慕课：5.1-5.4

X86处理器指令格式 (Instruction Format)

- x86处理器采用可变长度指令格式 (复杂指令集合)
- 操作码
 - 可选的指令前缀 (用于扩展指令功能)
 - 1 ~ 3字节的主要操作码
- 操作数
 - 可选的寻址方式域 (包括ModR/M和SIB字段)
 - 可选的位移量
 - 可选的立即数



学习指令的注意事项

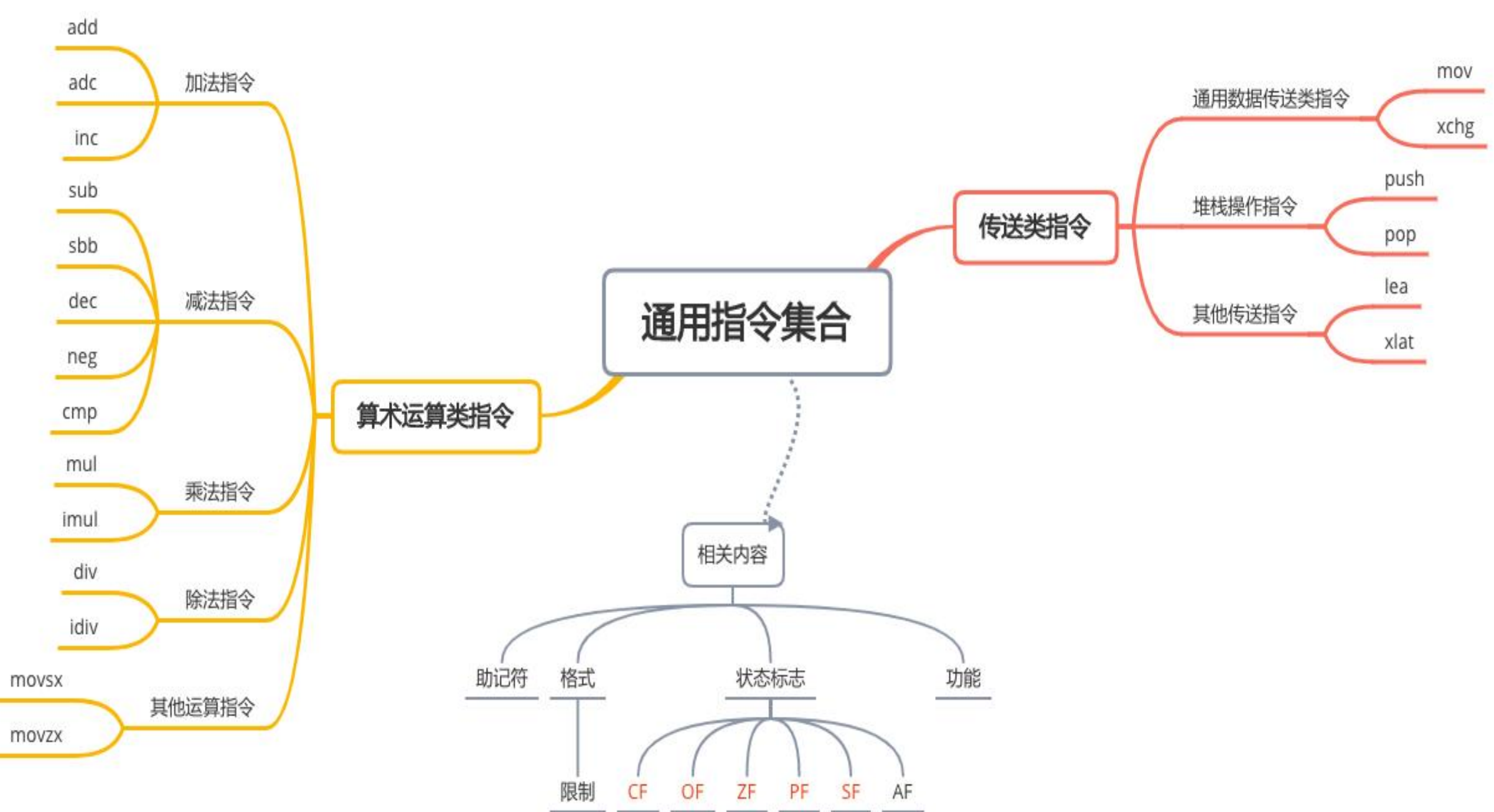
- **指令的功能**——该指令能够实现何种操作。通常指令助记符就是指令功能的英文单词或其缩写形式
- **指令支持的寻址方式**——该指令中的操作数可以采用何种寻址方式
- **指令对标志的影响**——该指令执行后是否对各个标志位有影响，以及如何影响
- **其他方面**——该指令其他需要特别注意的地方，如指令执行时的约定设置、必须预置的参数、隐含使用的寄存器等

✓ **MOV指令**

✓ **XCHG、LEA指令**

✓ **PUSH和POP指令**

✓ **算术运算指令**



- 慕课学习：
 - 视频：5.1-5.4
 - 单元测试：4.8

教学要点总结

- ✓ 寄存器
- ✓ 工作方式与存储模型
- ✓ 逻辑地址与物理地址转换
- ✓ 立即数寻址
- ✓ 寄存器寻址
- ✓ 存储器寻址